



# 大模型研讨课学习日志

## 系统视角理解大模型

作者: ZHY

时间: July 16, 2026

版本: 0.1



不要以为抹消过去，重新来过，即可发生什么改变。——比企谷八幡



# 目录

|                                   |   |
|-----------------------------------|---|
| 第 1 章 大模型研究史                      | 1 |
| 第 2 章 大模型基础知识                     | 2 |
| 2.1 整体概述                          | 2 |
| 2.2 数据准备                          | 2 |
| 2.2.1 数据清洗                        | 2 |
| 2.2.2 词元化                         | 2 |
| 2.2.2.1 词元化粒度的选择                  | 2 |
| 2.2.2.2 Byte Pair Encoding        | 3 |
| 2.2.3 从词元到索引                      | 3 |
| 2.3 词嵌入                           | 3 |
| 2.3.1 One-hot 编码                  | 3 |
| 2.3.2 Word2vec                    | 4 |
| 2.3.2.1 核心假设                      | 4 |
| 2.3.2.2 模型架构                      | 4 |
| 2.3.2.3 训练优化策略                    | 4 |
| 2.3.2.4 输入与输出嵌入矩阵                 | 4 |
| 2.3.2.5 Word2vec 的贡献              | 5 |
| 2.3.3 Transformer 中的 Embedding    | 5 |
| 2.4 位置编码                          | 5 |
| 2.4.1 Positional Encoding（绝对位置编码） | 5 |
| 2.4.2 RoPE（旋转位置编码）                | 6 |
| 2.5 本章小结                          | 6 |

# 第 1 章 大模型研究史

## 内容提要

### 语言学研究史

纵观数十载人类语言学发展史，语言学家们一直在探索人类语言的本质规律，经历了从行为主义到符号主义最总走向连接主义的过程。与人工智能的发展史一脉相承。  
关键的时间节点总结如下图所示。

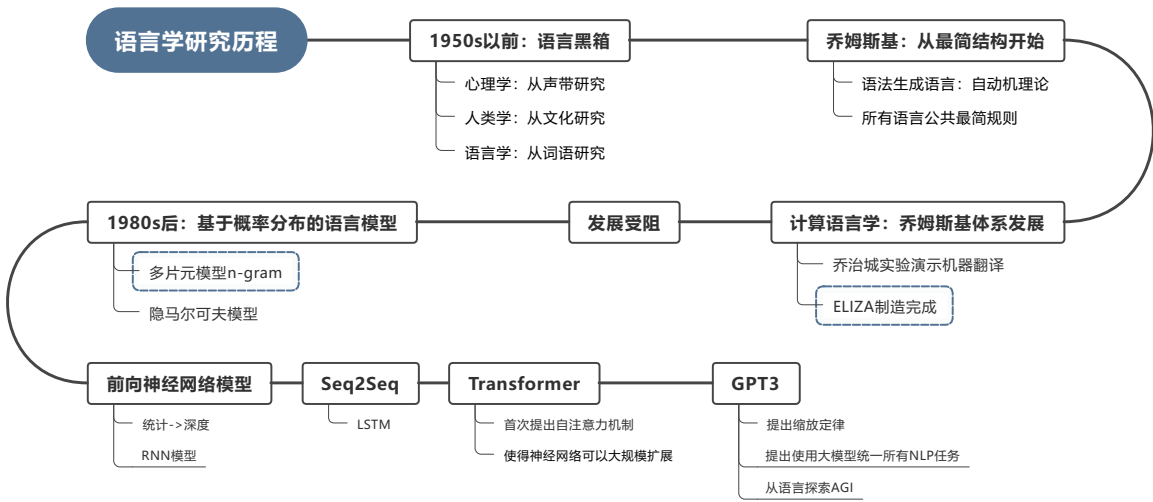


图 1.1: 语言学研究史

### 命题 1.1 (语言、语料与智能)

为什么科学家们总趋向认为语言是智能的核心？又为什么研究语言最终转向了大数据和大模型？

- 对信息的压缩即为智能。人类大脑在有限的神经元中存储和处理信息，并表达为语言，即智能。
- 人类语言的柯氏复杂度远远大于其他任何形式的符号系统。列举规则是不可行的。
- 语料可以反映人类语言系统的复杂规则，通过使用语料训练模型，可以一定程度模拟语言。

### 命题 1.2 (符号主义的语言学面临的困境)

- 通过训练语料得到的智能体，其智能不会超出人类的大脑智能。
- 应当深入大模型内部，试图寻找可能的规则推理。不应简单视作黑箱。

## 第2章 大模型基础知识

### 2.1 整体概述

大语言模型（Large Language Model, LLM）的本质是一个从离散文本到连续向量空间的映射函数。训练大模型的第一步，是将原始文本转化为神经网络能够理解的数值形式。本章沿着以下主线展开：

- 数据准备：**原始文本 → 清洗 → 词元化 → 训练数据；
- 词嵌入：**离散索引 → 稠密向量（One-hot → Word2vec → Transformer）；
- 位置编码：**为词向量注入序列位置信息（Positional Encoding → RoPE）。

上述三个环节层层递进，共同构成了大模型处理文本数据的基础设施。

### 2.2 数据准备

数据准备的目的是将原始语料转化为模型可用的数值输入。其核心流程为：

原始语料库 → 数据清洗 → 词元化 → 索引化 → 训练数据 (2.1)

**注** 数据准备的质量直接影响模型的上限。即使模型架构再先进，如果输入数据充满噪声或语义单位切分不当，模型的性能也会大打折扣。因此，数据准备常被视为大模型训练中最具工程挑战性的环节之一。

#### 2.2.1 数据清洗

数据清洗是预处理的第一步，旨在去除原始文本中的噪声，为后续词元化提供干净的输入。典型操作包括：

- 去除 HTML 标签、特殊控制字符和无关的格式化符号；
- 统一字符编码（如 Unicode 规范化），处理全角/半角转换；
- 过滤低质量、过短或重复的文本内容；
- 根据任务需要，决定是否移除停用词（在大模型语境下，通常保留停用词以维持完整语义）。

#### 2.2.2 词元化

词元化（Tokenization）是将清洗后的连续文本切分为基本语义单元（词元，Token）的过程。切分后的所有词元构成一个有限的列表，称为**词表（Vocabulary）**。词表是模型输出的空间基础——所有生成的内容都仅限于词表范围内的词元组合。

##### 2.2.2.1 词元化粒度的选择

根据切分粒度的不同，词元化方法分为三类，其对比见表 2.1。

表 2.1: 三种词元化粒度的对比

| 粒度             | 词表大小 | 序列长度 | 主要问题             |
|----------------|------|------|------------------|
| 字符级（Character） | 极小   | 极长   | 计算成本高，单字符语义弱     |
| 单词级（Word）      | 极大   | 适中   | 低频词利用率低，OOV 问题严重 |
| 子词级（Subword）   | 适中   | 适中   | 平衡语义与效率（主流方案）    |

子词级方法的核心优势在于：高频词拥有独立的完整词元，低频或未见过的词则被拆分为更常见的子词片段。这种机制有效缓解了**未登录词（Out-of-Vocabulary, OOV）**问题，同时将词表规模控制在合理范围内。

**命题 2.1 (OOV 问题的本质)**

未登录词是指不在预构建词表中的词元。当模型遇到 OOV 时，无法为其分配有效的向量表示，导致语义信息丢失、表示空间不完整、泛化能力下降。子词词元化通过将任意字符串拆解为词表中子词的组合，从根本上避免了“是否在词表中”的二元困境。

**2.2.2.2 Byte Pair Encoding**

BPE (Byte Pair Encoding) 是目前大模型 (GPT 系列、LLaMA 等) 中最广泛采用的子词词元化算法。其核心思想是迭代地合并语料中最频繁出现的相邻符号对。

**定理 2.1 (BPE 算法过程)**

BPE 通过以下步骤构建词表：

1. 初始化词表为所有单个字符 (或字节)；
2. 统计当前语料中所有相邻符号对的出现频率；
3. 将频率最高的一对符号合并为一个新的子词符号，并加入词表；
4. 将语料中所有该符号对替换为新的子词；
5. 重复步骤 2 至 4，直至词表规模达到预设要求。



**注** BPE 的本质是一种基于频率的贪心压缩算法。其合并操作是**不可逆**的——合并后的符号丢失了其组成成分的边界信息，因此同一文本在不同词表下会产生不同的词元序列。这是 BPE 的设计特性，而非缺陷：它不是在“恢复”某种客观的语言单位，而是在为特定语料“构造”一套最优的编码方案。

**2.2.3 从词元到索引**

神经网络无法直接处理字符或子词，必须将词元转换为数值索引。转换流程如下：

1. 建立从词元到词表索引的映射字典；
2. 将文本中的所有词元按顺序转换为对应的索引序列；
3. 在词表中添加特殊词元，辅助模型训练与推理：
  - [UNK]：表示未知词元，处理不在词表中的字符；
  - [PAD]：将不同长度的序列填充至相同长度，便于批处理；
  - [BOS] / [EOS]：标识序列的开始与结束；
  - [SEP]：分隔不同的句子或段落。

**2.2.4 词嵌入**

词嵌入 (Word Embedding) 的目标是将离散的词元索引转换为连续的稠密向量表示，使得语义信息能够在数学空间中被度量和计算。词嵌入方法经历了从 **One-hot** 到 **Word2vec** 再到 **Transformer** 端到端学习的演进历程。

**2.2.5 One-hot 编码**

One-hot 是最原始的词向量表示方法。

**定义 2.1 (One-hot 编码)**

对于大小为  $N$  的词表，任意词元  $w$  的 One-hot 向量  $\mathbf{v}_w \in \{0, 1\}^N$  定义为：若  $w$  在词表中的索引为  $i$ ，则  $\mathbf{v}_w[i] = 1$ ，其余维度均为 0。



**注** One-hot 编码的主要缺陷：

- (i) **维度灾难**：词表规模  $N$  通常高达数十万，向量极度稀疏，计算效率低下；

- (ii) **语义孤立**: 任意两个不同的 One-hot 向量正交, 余弦相似度恒为 0, 无法刻画“猫”与“狗”之间的语义关联。

## 2.2.6 Word2vec

Word2vec 是 Google 于 2013 年提出的里程碑式词嵌入方法。它通过无监督学习从大规模语料中训练出稠密的、低维的实值向量, 彻底改变了词的表示方式。

### 2.2.6.1 核心假设

Word2vec 建立在分布式语义假设之上:

#### 命题 2.2 (分布式语义假设)

文本中离得越近的词语, 其语义相似度越高。词的语义由其上下文决定——“You shall know a word by the company it keeps” (Firth, 1957)。

### 2.2.6.2 模型架构

Word2vec 包含两种互为镜像的模型架构, 其结构差异决定了各自的适用场景:

- **CBOW (连续词袋模型)**: 根据上下文词预测当前中心词。将上下文词向量求平均后通过 Softmax 输出预测概率。训练速度快, 适合大规模语料, 但对低频词效果一般。
- **Skip-gram (跳元模型)**: 根据当前中心词预测其上下文词。每个上下文词独立预测, 为低频词提供更多训练信号, 效果好但训练速度较慢 (约为 CBOW 的 3-5 倍)。

两者的核心差异可归纳为表 2.2。

表 2.2: CBOW 与 Skip-gram 的对比

| 对比维度     | CBOW      | Skip-gram |
|----------|-----------|-----------|
| 输入 → 输出  | 上下文 → 中心词 | 中心词 → 上下文 |
| 训练样本数/窗口 | 1 个       | 2k 个      |
| 训练速度     | 快         | 慢         |
| 低频词效果    | 一般        | 好         |
| 适用场景     | 大规模语料     | 中小规模语料    |

### 2.2.6.3 训练优化策略

Word2vec 的输出层需要计算 Softmax, 其分母需要对词表中所有  $V$  个词求和, 复杂度为  $O(V)$  (通常数十万), 计算代价极高。为此引入了两种优化策略:

- **层次化 Softmax**: 将多分类问题转化为二叉树上的若干二分类问题, 复杂度由  $O(V)$  降至  $O(\log V)$ ;
- **负采样**: 将多分类问题简化为区分目标词和噪声词的二分类问题, 每个训练样本只需计算  $K+1$  个词 ( $K$  通常为 5-20), 复杂度降至  $O(K)$ 。

### 2.2.6.4 输入与输出嵌入矩阵

Word2vec 同时维护两个嵌入矩阵:

- 输入嵌入矩阵  $\mathbf{W} \in \mathbb{R}^{V \times d}$ : 每一行是一个词的输入向量 (作为中心词时的表示), 训练完成后保留为最终词向量;
- 输出嵌入矩阵  $\mathbf{W}' \in \mathbb{R}^{d \times V}$ : 每一列是一个词的输出向量 (作为上下文词时的表示), 训练后丢弃。

**注**[为什么需要两个矩阵?] 同一词在模型中扮演“中心词”和“上下文词”两种角色。若共用同一套参数，输入侧和输出侧的梯度更新会相互冲突；分离两套参数使各自独立优化，同时增加模型容量。训练完成后， $\mathbf{W}$  的每一行即为对应词的稠密向量。

### 2.2.6.5 Word2vec 的贡献

#### 命题 2.3 (Word2vec 的历史地位)

Word2vec 证明了语义信息可以通过基于共现的预测任务被有效地编码进低维向量空间中。其词类比现象（如  $\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx \mathbf{v}_{\text{queen}}$ ）成为评估词向量质量的经典基准。这一思想直接启发了后续 ELMo、BERT 及当代大语言模型。

### 2.2.7 Transformer 中的 Embedding

在 Transformer 架构中，词嵌入不再依赖外部预训练模型（如 Word2vec），而是作为模型的一部分进行端到端自学习。

#### 定理 2.2 (Transformer 嵌入层)

Transformer 的嵌入层本质上是一个可训练的线性变换矩阵  $\mathbf{E} \in \mathbb{R}^{V \times d}$ 。给定词元的 One-hot 向量  $\mathbf{o} \in \mathbb{R}^V$ ，其嵌入向量为：

$$\mathbf{e} = \mathbf{o}^\top \mathbf{E} \quad (2.2)$$

该变换将离散的 One-hot 索引映射为连续的稠密向量，整个过程通过梯度反向传播进行优化。

**注**[Word2vec 与 Transformer 嵌入的本质区别]

- **Word2vec**: 静态表示。训练完成后词向量固定不变，“苹果”在“吃苹果”和“苹果公司”中向量完全相同，无法处理一词多义。
- **Transformer**: 动态的、上下文相关的表示。同一词在不同上下文中经自注意力机制融合全局信息后产生不同向量；配合位置编码可捕捉序列顺序信息。

从“静态”到“动态”，是词表示技术发展的核心范式转变。

## 2.3 位置编码

Transformer 的自注意力机制本身是置换不变的（即打乱输入顺序，输出不变），但语言的语义强烈依赖词序。因此需要显式地将位置信息注入模型。

### 2.3.1 Positional Encoding（绝对位置编码）

绝对位置编码通过在输入词向量上叠加位置向量来注入位置信息。

#### 定义 2.2 (正弦-余弦位置编码)

对于位置  $pos$  和维度  $i$ ，位置编码定义为：

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right) \quad (2.3)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right) \quad (2.4)$$

其中  $d$  为嵌入维度。位置编码维度与词向量相同，两者相加后输入模型。

**注**[绝对位置编码的特性]

- **绝对位置信息**：不同位置使用不同频率的波长表示，可唯一标识每个位置；
- **相对位置关系**：正弦-余弦函数的线性变换性质使得  $PE_{pos+k}$  可表示为  $PE_{pos}$  的线性函数，从而可捕获相对位置关系；
- **局限性**：位置编码是预定义的固定值，且受限于训练时的最大序列长度，无法泛化到更长的序列。

### 2.3.2 RoPE（旋转位置编码）

RoPE (Rotary Position Embedding) 将位置信息直接编码到自注意力的计算过程中，是当前大模型（如 LLaMA、Qwen 等）的主流位置编码方案。

#### 定理 2.3 (RoPE 的核心思想)

RoPE 利用旋转矩阵对查询向量  $\mathbf{q}$  和键向量  $\mathbf{k}$  进行旋转变换，使得内积结果  $\mathbf{q}_m^\top \mathbf{k}_n$  仅依赖于相对位置  $m - n$ ：

$$f(\mathbf{q}_m, m)^\top f(\mathbf{k}_n, n) = g(\mathbf{q}_m, \mathbf{k}_n, m - n) \quad (2.5)$$

其中  $f$  为带位置编码的旋转变换， $m$  和  $n$  为位置索引。



注[RoPE 的优势]

- **灵活应对不同序列长度**：无绝对位置上限，可外推到训练时未见过的更长序列；
- **有效捕获相对位置**：注意力分数天然依赖于词之间的相对距离，符合语言建模的直觉（距离越近的词通常相关性越强）；
- **无需额外参数**：旋转矩阵是确定性的，不引入可训练参数，不增加模型容量。

## 2.4 本章小结

本章系统梳理了大语言模型数据准备与词表示的全链路，核心要点总结如下：

1. **数据准备**：原始文本经清洗、BPE 词元化、索引化后转化为模型输入。子词词元化通过将任意文本拆解为词表中子词的组合，从根本上解决了 OOV 问题。
2. **词嵌入**：词表示从 One-hot 的稀疏离散表示，演进到 Word2vec 的静态稠密表示，再到 Transformer 中端到端学习的动态上下文表示。这一演进的核心驱动力是“让词向量承载更丰富的语义信息”。
3. **位置编码**：为弥补自注意力机制置换不变性的缺陷，通过绝对位置编码（Transformer）或旋转位置编码（RoPE, LLaMA 等）将词序信息注入模型。RoPE 因其良好的长度外推能力成为当代大模型的主流选择。上述环节共同构成了大模型理解与生成语言的基础设施，是深入理解大模型工作原理的必备知识。